GPU MEMORY ARCHITECTURE DESIGN

Contents

Introduction 2

GPU Memory Architecture Features 3

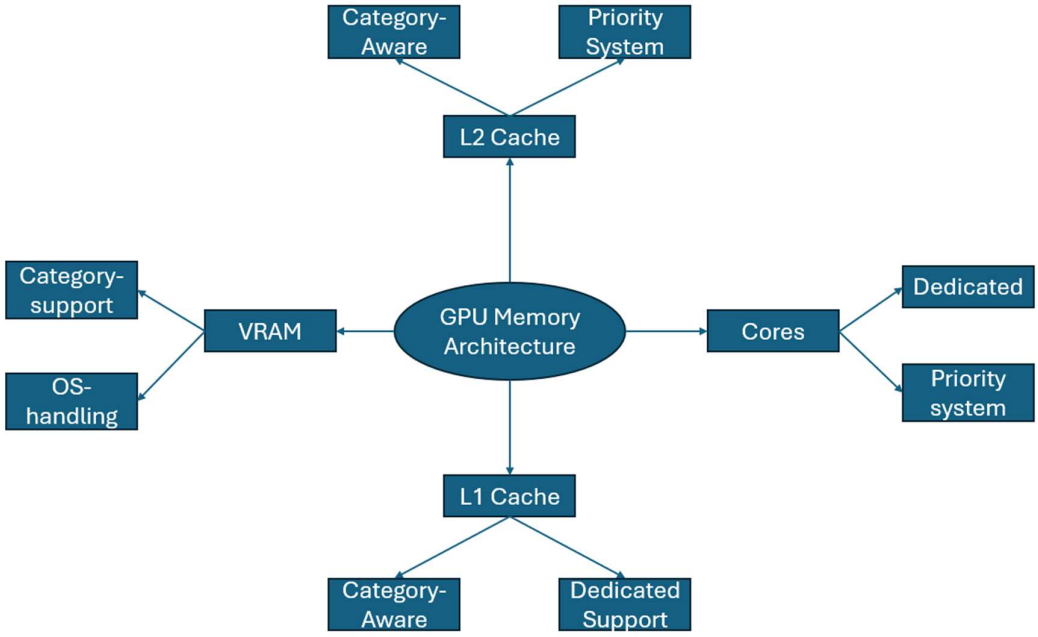
GPU Memory Architecture Workflow 4

GPU Memory Architecture Trade-offs 5

Introduction

This GPU Memory Architecture is designed to improve performance of GPUs in modern computing systems. This was also developed to address the current limitations of GPU components such as VRAM and Cache. This GPU memory architecture was developed as current GPUs aren't designed to handle the heavy workloads that modern systems especially AI and gaming generate. This GPU memory architecture design will explain the changes made to VRAM, Cache and Cores. It will explain the workflow and the trade-offs of the design.

This GPU Memory Architecture cannot promise perfect performance or handling of workloads. However, it can promise to improve performance and handling of workloads compared to current GPU memory architecture. It is important to remember that this is just a design, and engineers may face more challenges than defined within this specification document. This GPU Memory architecture should also be adapted based on the system that it is used for. For example, gaming systems may prioritise apps over the operating system.



The spider diagram shows a visual representation of the GPU memory architecture. It shows two features of all 4 components involved in the architecture. This spider diagram does not show the exact details of how these features or systems work. More detail on the architecture can be found in the sections below.

GPU Memory Architecture Features

- **This GPU memory architecture will discuss changes to VRAM, L2 & L1 cache, and Cores/SMs starting with VRAM.**
- **VRAM is used to store data such as 3D Geometry, Images and Rendered Frames for the GPU to fetch and then execute using other components.**
- **VRAM data should be organised into specific categories such as models, textures and frame buffers to make access times quicker.**
- **These categories can be sorted out by OS or the GPU's memory controller through an VRAM space allocation system.**
- **This system will calculate the storage for frame buffers by using screen resolution and calculating 10% of that storage and adding as a safety buffer**
- **For the models and textures category, the OS will provide each of them with 20% of space within VRAM after the frame buffers category is calculated.**
- **The OS will then create a free space category with the remaining space within VRAM which the models and textures categories can take from.**
- **It is important to remember that after the category no longer requires the space then it will be placed back into the free space category.**
- **To decide which category has priority in free space, the OS will calculate the size of data within category and allocate space as needed.**
- **For the case that VRAM has no available space, the GPU can fallback onto the app category of system RAM.**
- **The GPU will use a hybrid system between fixed and dynamic to manage categories in L2 cache, L1 cache and SMs/CUs.**
- **The GPU will need to allocate a fixed number of cores and space within L2 cache for the frame buffer category.**
- **This is because frame buffers are latency critical and cannot afford to be starved out by a dynamic priority system.**
- **The GPU can then use a dynamic priority system to allocate the remaining cores to the textures and model categories.**
- **This means that some cores would be allocated to models while more cores could be allocated to textures depending on workload.**
- **The OS can calculate workload based on CPU load, RAM load, network usage, thermals, power state, Disk I/O, and GPU utilization.**
- **For results, the GPU should directly write from registers to VRAM instead of writing to multiple caches.**
- **This is important because writing back would have to go down the whole chain slowing it down significantly.**
- **For reading, the GPU should fetch from VRAM, Put the data in L2 and L1 caches and finally the cores execute what's needed.**
- **This GPU architecture requires logical and OS changes no physical hardware changes.**

GPU Memory Architecture Workflow

- **This workflow will help anyone trying to create this design to understand how the process works.**
- **It is also important to know that this workflow uses the dynamic priority system workflow instead of fixed.**
- **First, data is loaded from RAM by the CPU into VRAM.**
- **The OS organises the data loaded from RAM into the categories models, textures and frame buffers.**
- **For example, the user is playing GTA 6 which demands heavy physics workloads.**
- **Second, the most frequently used data of the models category is placed into L2 cache by the priority system.**
- **The other two categories, textures and frame buffers are allocated less L2 cache space by the priority system.**
- **Third, the most frequently used data of each category is separated into the dedicated L1 cache of each SM/CU.**
- **This is done by the priority system as most L1 caches will be dedicated to the models category.**
- **Fourth, each SM/CU is dedicated to a specific category by the priority system based on the heaviest workload.**
- **This means that most SM/CU would be dedicated to the models category as it would be the heaviest workload produced by GTA 6.**
- **Fifth, data is loaded into registers per thread for SM/CU to access easily without heading to cache.**
- **The SMs/CUs would then execute the instructions or tasks needed using the data stored in registers, cache and even VRAM if needed.**
- **Sixth, results would be written directly to VRAM after they are produced.**
- **This means that from the registers the GPU writes the results directly to VRAM instead of writing through each cache.**
- **This is done by the GPU VRAM bus which directly writes the results to VRAM instead of passing through multiple caches.**
- **Finally, the results are displayed on the user's interface of GTA 6.**
- **This means that the user will be able to see the physics models and other assets in the game once the results have been delivered and processed.**
- **Make sure for certain systems such as consoles or embedded systems this workflow is adapted for the fixed priority system.**

GPU Memory Architecture Trade-offs

- **This GPU Memory Architecture has trade-offs just like any other system I have created.**
- **I have identified 4 main trade-offs with this GPU Memory Architecture.**
- **Engineers may also face additional challenges during actual implementation of this architecture as I cannot test implementation trade-offs.**
- **The first trade-off is Heat.**
- **This trade-off exists because the GPU can do more work more quickly which can produce heat.**
- **This can cause the GPU to overheat especially in hot environments and long device usage hours.**
- **This trade-off can be managed through cooling systems such as heat sinks and fans to manage the heat.**
- **The second trade-off is Latency.**
- **This trade-off exists because of the priority system real-time changes to multiple components in the GPU.**
- **This means that the user may experience a lag spike as SMs/CUs, and cache must change categories and tasks even if it is a quick process.**
- **This trade-off can be managed through using a fixed priority system where possible and using a combination of hardware-level and OS-level management.**
- **The third trade-off is Time-Consumption.**
- **This trade-off exists because creating a new GPU design using this architecture can take years even if there are no direct hardware changes.**
- **This is because OS vendors and hardware engineers would need to coordinate for this GPU Memory architecture to exist.**
- **It is also because this GPU memory architecture may have to be adapted to multiple different systems.**
- **This trade-off can be managed through structured development processes and business agreements.**
- **It is also acceptable as new generations of GPU usually can be time-consuming to create in general.**
- **The fourth trade-off is OS-Reliance**
- **This trade-off exists because the OS must manage the sorting of data into VRAM categories and the real-time changes with the priority system.**
- **This means that the memory organisation of VRAM and the effectiveness of the priority system can be limited by OS limitations.**
- **This trade-off can be managed through hardware-level scheduling and pre-existing GPU systems that don't rely on the OS.**
- **This trade-off is already acceptable as operating systems manage VRAM and priority systems in modern computing systems.**